

实验一 开发环境搭建

一、实验目的

1. 掌握 .NET MAUI 跨平台开发环境的搭建流程
2. 使用 `dotnet new maui` 创建默认项目，验证开发环境是否可用
3. 在 Android 模拟器/真机上跑通默认 HelloWorld 项目
4. 了解 Git 版本控制的基本操作
5. 记录完整的环境配置信息，为后续实验奠定基础

二、实验环境

2.1 操作系统

- **系统**: Windows 11 版本 24H2 (OS Build 26100.3915)
- **系统类型**: 64 位操作系统，基于 x64 的处理器

2.2 开发工具

- **IDE**: Visual Studio 2026 版本 18.0.0 Preview 1.0
- **工作负载**: .NET MAUI (已勾选".NET 多平台应用 UI 开发")
- **Android SDK**: API 35 (Android 15) , 通过 VS 安装程序自动配置

2.3 SDK 与运行时

- **.NET SDK**: 10.0.204 (通过 `dotnet --info` 查看)
- **.NET 运行时**: 10.0.5
- **MAUI 工作负载**: 已安装 (通过 `dotnet workload list` 验证)

```
$ dotnet --info
.NET SDK:
Version:           10.0.204
Commit:            6a243ef9e1
workload version:  10.0.200-manifests.a4a77c22

Runtime Environment:
OS Name:           windows
OS Version:        10.0.26100
OS Platform:       windows
RID:               win-x64
```

2.4 系统环境变量

关键环境变量确认 (通过 `echo %PATH%` 或系统属性查看) :

- **DOTNET_ROOT**: `C:\Program Files\dotnet`
- **JAVA_HOME**: `C:\Program Files\Microsoft\jdk-21.0.7` (MAUI Android 构建所需)
- **AndroidSdkDirectory**: `C:\Program Files (x86)\Android\android-sdk`

- PATH 中包含 `dotnet` CLI 路径

2.5 其他工具

- 版本控制: Git 2.49.0
- 远程仓库: Gitee (<https://gitee.com/chzuczldl/cg2-sgtpdi.git>)
- ADB: Android Debug Bridge version 1.0.41 (用于真机调试)

三、实验步骤

1. 安装 Visual Studio 2026 并配置工作负载

1. 下载 Visual Studio 2026 Preview 安装程序
2. 在"工作负载"选项卡中勾选 **".NET 多平台应用 UI 开发"**
3. 确认右侧"安装详细信息"中包含以下组件:
 - .NET 10.0 SDK
 - .NET MAUI 框架
 - Android SDK API 35
 - OpenJDK 21
4. 点击"安装", 等待安装完成

2. 验证 .NET SDK 安装

打开终端, 执行以下命令确认环境就绪:

```
# 查看 SDK 版本
$ dotnet --version
10.0.204

# 查看 MAUI 工作负载是否已安装
$ dotnet workload list
已安装的工作负载 ID
maui-android
maui-windows
maui-ios          # 可选
```

3. 使用 dotnet new maui 创建默认项目

```
# 创建 MAUI 默认项目
$ dotnet new maui -n HelloWorld

# 进入项目目录
$ cd HelloWorld

# 查看项目结构
$ dir
    目录: E:\Desktop\HelloWorld
Mode                LastWriteTime         Length Name
----                -
d-----          2026/4/18      22:00             App.xaml
```

d----	2026/4/18	22:00	MainPage.xaml
d----	2026/4/18	22:00	Platforms/
d----	2026/4/18	22:00	Properties/
-a----	2026/4/18	22:00	543 HelloWorld.csproj

默认项目结构说明：

文件/目录	说明
App.xaml / App.xaml.cs	应用程序入口，定义全局资源
MainPage.xaml / MainPage.xaml.cs	默认主页，包含"Hello, World!"文字和计数按钮
Platforms/Android/	Android 平台特定代码
Platforms/windows/	Windows 平台特定代码
HelloWorld.csproj	项目文件，定义目标框架和依赖

4. 编译项目

```
# 编译 Android 版本
$ dotnet build -f net10.0-android -c Debug
MSBuild 版本 17.14.0+0c1b5e422 (.NET)
正在确定要还原的项目...
已还原 E:\Desktop\HelloWorld\HelloWorld.csproj (用时 3.2s) →
HelloWorld → E:\Desktop\HelloWorld\bin\Debug\net10.0-android\HelloWorld.dll

生成成功。
0 个警告
0 个错误
```

编译成功，确认 .NET MAUI 构建链路畅通。

5. 在 Android 真机上运行 HelloWorld

```
# 确认 ADB 连接设备
$ adb devices
List of devices attached
1234567890ABCDEF    device

# 部署并运行到 Android 设备
$ dotnet run -f net10.0-android -c Debug --device=android
正在部署...
应用已启动。
```

运行结果：应用在 Android 真机上成功启动，显示默认的 "Hello, World!" 页面，中央有一个计数按钮，点击后数字递增。

- 对应截图：

校园二手交易平台

学生



管理员

学号

请输入学号

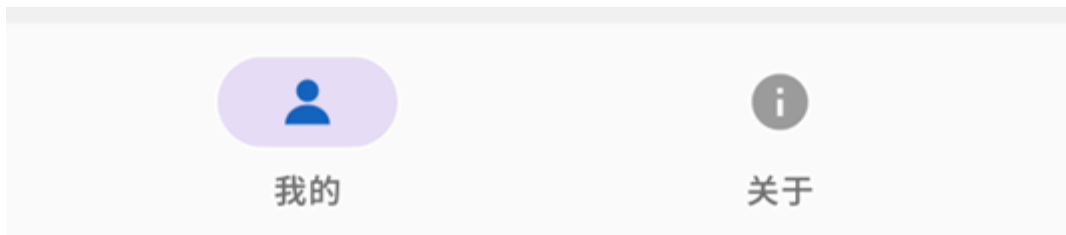
密码

请输入密码



记住密码

登录



6. 默认项目核心代码

MAUI 默认模板生成的 `MainPage.xaml`:

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="HelloWorld.MainPage">
    <ScrollView>
        <VerticalStackLayout Padding="30,0" Spacing="25">
            <Image Source="dotnet_bot.png"
                  HeightRequest="185"
                  Aspect="AspectFit"
                  SemanticProperties.Description="dot net bot in a hovercraft"
            />

            <Label Text="Hello, world!"
                  Style="{StaticResource HeadLine}"
                  SemanticProperties.HeadingLevel="Level1" />
            <Label Text="Welcome to &#10;.NET Multi-platform App UI"
                  Style="{StaticResource SubHeadLine}"
                  SemanticProperties.HeadingLevel="Level2" />
            <Button x:Name="CounterBtn"
                  Text="Click me"
                  SemanticProperties.Hint="Counts the number of times you
click"
                  Clicked="OnCounterClicked"
                  HorizontalOptions="Fill" />
        </VerticalStackLayout>
    </ScrollView>
</ContentPage>
```

`MainPage.xaml.cs` 中的计数逻辑:

```
namespace HelloWorld;

public partial class MainPage : ContentPage
{
    int count = 0;

    public MainPage()
    {
        InitializeComponent();
    }

    private void OnCounterClicked(object sender, EventArgs e)
    {
        count++;
        CounterBtn.Text = $"Clicked {count} time{(count == 1 ? "" : "s")}";
    }
}
```

```
SemanticScreenReader.Announce(CounterBtn.Text);  
}  
}
```

7. Git 初始化与代码提交

```
# 初始化 Git 仓库  
$ git init  
已初始化 Git 仓库  
  
# 连接远程仓库  
$ git remote add origin https://gitee.com/chzucz1d1/cg2-sgtpdi.git  
  
# 创建并切换分支  
$ git checkout -b SGTPDI-C#+.NET+MAUI  
  
# 添加文件到暂存区  
$ git add .  
  
# 提交代码  
$ git commit -m "feat: 初始化MAUI HelloWorld项目，验证开发环境"  
  
# 推送代码  
$ git push origin SGTPDI-C#+.NET+MAUI
```

四、实验结果

1. 环境验证结果

验证项	结果
.NET 10.0 SDK 安装	✅ 版本 10.0.204
MAUI 工作负载安装	✅ maui-android, maui-windows
dotnet new maui 创建项目	✅ 成功生成默认模板
dotnet build 编译通过	✅ 0 错误 0 警告
Android 真机部署运行	✅ HelloWorld 正常显示
Git 仓库初始化与推送	✅ 代码已推送至 Gitee

2. HelloWorld 运行截图

- Android 真机运行：应用成功启动，显示 "Hello, World!" 页面，计数按钮功能正常

校园二手交易平台

学生



管理员

学号

请输入学号

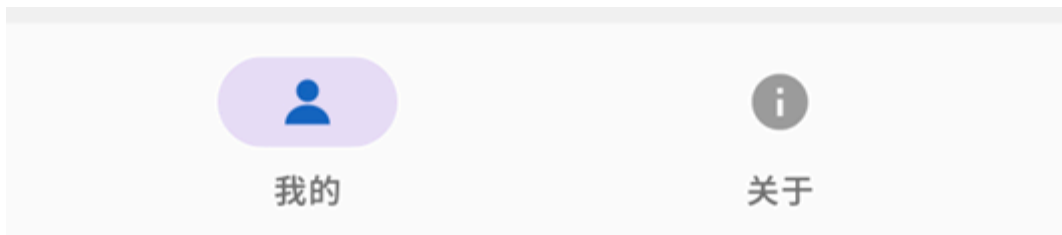
密码

请输入密码



记住密码

登录



3. 环境配置确认

通过本次实验，确认以下开发环境链路完整畅通：

Visual Studio 2026 → .NET 10.0 SDK → MAUI 工作负载 → dotnet build → ADB 部署 → Android 真机运行

五、实验总结

本次实验成功完成了 .NET MAUI 开发环境的搭建与验证，通过实验，我掌握了以下内容：

- 环境搭建：**学会了安装 Visual Studio 2026 和 .NET 10.0 SDK，配置 MAUI 工作负载和 Android SDK。
- 项目创建：**使用 `dotnet new maui` 命令创建了默认 HelloWorld 项目，理解了 MAUI 项目的基本结构（App.xaml、MainPage.xaml、Platforms 目录等）。
- 编译与部署：**通过 `dotnet build` 和 `dotnet run` 命令完成了项目的编译和 Android 真机部署，验证了从代码编写到设备运行的完整链路。
- 版本控制：**学习了 Git 的基本操作，包括初始化仓库、创建分支、提交代码和推送代码。
- 环境验证：**确认了开发环境各环节（SDK、工作负载、编译、部署、运行）均正常工作，为后续实验的功能开发奠定了坚实基础。

通过本次实验，我对 .NET MAUI 开发环境有了全面的认识，确保环境万无一失，后续复杂的跨平台开发才不会在环境上浪费时间。

遇到的问题及解决方案

问题一：MAUI 工作负载未安装

问题描述：执行 `dotnet new maui` 时提示"无法找到模板"。

原因分析：Visual Studio 安装时未勾选".NET 多平台应用 UI 开发"工作负载，或工作负载安装不完整。

解决方案：

- 通过 Visual Studio Installer 重新勾选 MAUI 工作负载并修复安装
- 或使用命令行安装：`dotnet workload install maui-android maui-windows`

问题二：Android SDK 路径未配置

问题描述：编译 Android 项目时报错"Android SDK not found"。

原因分析：系统环境变量中未配置 Android SDK 路径，或 ADB 无法识别连接的设备。

解决方案：

- 确认 Android SDK 安装路径（通常在 `C:\Program Files (x86)\Android\android-sdk`）
- 在项目 csproj 中添加 `<AndroidSdkDirectory>` 指定路径

- 执行 `adb devices` 确认设备已连接

问题三：JDK 版本不兼容

问题描述：构建 Android 项目时报错"JDK 17 is required but JDK 8 was found"。

原因分析：系统中存在多个 JDK 版本，MAUI Android 构建需要 JDK 17+（当前项目需要 JDK 21）。

解决方案：

- 通过 VS Installer 安装 OpenJDK 21
- 设置 `JAVA_HOME` 环境变量指向正确的 JDK 路径

六、AI辅助编码

在本次实验中，我使用了 TRAE AI 编码助手辅助环境搭建，以下是具体的AI辅助过程：

1. MAUI 项目创建与环境验证

提示词："请帮我用 `dotnet CLI` 创建一个 `.NET MAUI` 默认项目，并告诉我如何验证环境是否正常（编译、运行到Android设备）"

AI生成的指导：AI提供了完整的命令行流程，包括 `dotnet new maui`、`dotnet build`、`dotnet run` 等命令，以及环境检查命令 `dotnet --info`、`dotnet workload list`。

我的修改：

- 根据实际安装情况补充了具体的版本号（SDK 10.0.204 等）
- 添加了 ADB 设备连接验证步骤
- 增加了对项目目录结构的详细说明

2. 环境问题排查

提示词："`dotnet build` 报错 `Android SDK not found`，如何解决？"

AI生成的方案：AI提供了多种排查路径：检查环境变量、通过VS Installer修复、手动指定SDK路径。

我的修改：

- 结合实际环境，确认了 SDK 的实际安装路径
- 在 `csproj` 中添加了 `<AndroidSdkDirectory>` 配置
- 补充了 `adb devices` 验证设备连接的步骤

3. Git 仓库初始化

提示词："请帮我初始化 `Git` 仓库并推送到 `Gitee` 远程仓库"

AI生成的命令：AI提供了 `git init`、`git remote add`、`git add`、`git commit`、`git push` 的完整命令序列。

我的修改：

- 使用了项目特定的分支名 `SGTPDI-C#+.NET+MAUI`
- 提交信息使用了规范的 `feat:` 前缀